Links

Was ist ein Link?

Ein HTML-Element <a> mit href ist ein Hyperlink:

```
<a id="L01-menu" href="/02-buttons.html">Next lesson: buttons</a>
```

href bezeichnet das Ziel, id identifiziert das Element und der Inhalt zwischen den Tags beschriftet den Link. href="#help" verweist auf einen Abschnitt derselben Seite. target="_blank" öffnet normalerweise einen neuen Tab oder ein neues Fenster. download fordert einen Download an. Ein <a> ohne href besitzt nicht automatisch die Link-Semantik und wird möglicherweise nicht über link.text gefunden.

Benachbarte Links bleiben getrennte DOM-Elemente, auch wenn ihre Texte optisch zusammenlaufen. Ein
 oder ein eigener Absatz verbessert die Darstellung, ohne den Locator zu verändern.

Sichtbarer Text und zugänglicher Name

```
link:
  text: 'Next lesson: buttons'
  exact: true
```

AGATE sucht hier nach der Rolle LINK und dem zugänglichen Namen, dem „Accessible Name“. Dieser entspricht häufig dem sichtbaren Text. Attribute wie aria-label oder aria-labelledby sowie Bildalternativen können ihn verändern:

```
<a href="/help" aria-label="Dokumentation öffnen">Hilfe</a>
```

Hier sollte link.text: 'Dokumentation öffnen' verwendet werden. Alternativ eignet sich eine stabile ID. link.text durchsucht nicht einfach den HTML-Quelltext.

Verschachtelte Elemente

```
<a id="L01-help" href="#help"><span>(H)elp</span></a>
```

span ist ein Inline-Container. Sein Text kann zum Namen des übergeordneten Links beitragen. Der passende Locator bleibt link.text: '(H)elp'; ein separater Span-Locator ist nicht notwendig. Dasselbe gilt für typische Formatierungselemente wie <b> und .

```
<a href="/help"><span>Help</span><span> overview</span></a>
```

Der Name lautet normalerweise Help overview. Ohne Leerzeichen zwischen den Textteilen muss HTML kein zusätzliches Leerzeichen einfügen. Daher tatsächlichen DOM-Inhalt und Namen prüfen.

Eine Grafik mit alt="Hilfe" kann den Linknamen beeinflussen. Dekorative Bilder mit leerem alt sollen keinen Namen beitragen. aria-hidden="true" kann dekorative Inhalte aus der

Namensberechnung ausschließen. Bei versteckten Elementen, ARIA-Beziehungen oder ungewöhnlichen Symbolen nicht allein vom Screenshot auf den Namen schließen.

**Leerzeichen und **

Die korrekte Schreibweise ist , nicht &nsp;. Der Browser wandelt diese HTML-Entität in das geschützte Leerzeichen U+00A0 um.

```
<a href="/help">  (H)elp  </a>
```

Bei der Auswahl über den zugänglichen Namen werden äußere Leerzeichen normalisiert. exact: true verlangt deshalb keine identischen Leerzeichen im HTML-Quelltext. Auch mehrere gewöhnliche Leerzeichen innerhalb eines Namens werden normalisiert.

Ein Zero-Width-Zeichen wie U+200B ist kein NBSP. Nicht annehmen, dass alle unsichtbaren Zeichen entfernt werden. Bei solchen Fällen die Zeichen untersuchen oder eine stabile ID verwenden.

CSS-Attributvergleiche funktionieren anders: input[value="(O)K"] prüft den Attributwert und erhält nicht automatisch die Namensnormalisierung. Locator-Normalisierung und die Normalisierung einer TEXT_GLOB-Prüfung sind außerdem unterschiedliche Funktionen.

Parameter eines Testschritts

Parameter	Typ	Pflicht / Standard	Bedeutung
type	String	Pflicht	GUI.
op	String	Explizit angeben	EXEC für Aktionen, ASSERT für Prüfungen.
action	String	Pflicht bei der beschriebenen action-Form	Implementierte Werte mit op EXEC oder ASSERT: siehe die vollständige Aktionsreferenz unten.
id	String	Optional, empfohlen	Stabile Kennung des Testschritts im Log.

page	String	Optional, empfohlen	Seitenbeschreibung im Log; öffnet oder wählt keinen Tab.
timeout	Integer	Optional	Millisekunden; sonst gilt die vorhandene Schritt-/Suite-/Serverkonfiguration.
link	Objekt	Alternative zu target	Vereinfachter Link-Locator. Nicht mit target, selector oder anderen vereinfachten Elementen kombinieren.
target	Objekt	Alternative zu link	Technischer Locator mit genau einer Strategie.
expected	String/Integer	Abhängig von der Prüfung.	Erwarteter Text oder Anzahl; kein CLICK-Argument.

Implementierte Aktionen für Links

Die folgende Liste ist abschließend für die hier beschriebene Verarbeitung eines Link-Locators im geprüften Serverstand. op bestimmt die Bedeutung von action: Bei EXEC wird eine Aktion ausgeführt, bei ASSERT eine Prüfung. Der gleiche Locator kann für unterschiedliche Aktionen verwendet werden.

op: EXEC — Aktionen am Link

action	Funktion	Zusätzliche Parameter	Bedingungen

CLICK	Klickt den Link mit automatischer Prüfung seiner Ausführbarkeit.	Keine weiteren Pflichtparameter.	Ein eindeutiger, sichtbarer und ausführbarer Treffer; Navigation oder Popup hängt von HTML ab.
HOVER	Bewegt den Mauszeiger auf den Link.	Keine weiteren Pflichtparameter.	Für Hover-Menüs oder Tooltip-Verhalten; danach den ausgelösten Zustand prüfen.
PRESS	Sendet eine Taste/Tastenkombination an den Link.	value erforderlich, etwa Enter oder Tab.	Enter kann einen echten Link aktivieren; Tastaturverhalten hängt vom Element ab.
WAIT	Wartet auf den Zustand des Link-Locators.	state optional: attached, detached, visible, hidden; Standard visible. timeout optional.	Mit link ist dies eine Zustandswarteoperation. value ist hier keine Lernpause.
EXTRACT	Liest Text oder ein Attribut und speichert den Inhalt in einem Buffer.	attribute optional; buffer optional, alternativ response.	Mit attribute z. B. href lesen; ohne attribute innerText. Standardbuffer extracted. Fehlendes Attribut führt zu einem Fehler.
DOWNLOAD	Klickt und wartet auf ein echtes	value optional als Dateiname;	Ein normaler Navigationslink

	Browser-Downloadereignis; speichert die Datei.	buffer optional, Standard download; timeout optional.	oder inline angezeigtes PDF ist nicht automatisch ein Download. Dateipfad richtet sich nach dem Session-Ausgabeverzeichnis.
DOWNLOAD_PDF	Speichert den originalen PDF-Antwortinhalt des ausgelösten POST und schließt neu entstandene Popups.	outputFile und responseUrlGlob erforderlich; buffer optional, Standard download; timeout optional; ignoreHttpsErrors optional.	Spezielle implementierte POST-PDF-Verarbeitung; kein universeller GET-PDF-Downloader. PDF-Antwort muss zum Muster passen. TLS-Prüfung nur gezielt konfigurieren.

EXTRACT mit source: value ist zwar im allgemeinen GUI-Code vorhanden, aber für einen gewöhnlichen Link ungeeignet: Ein Link ist kein Eingabefeld. Für Links attribute oder den Standard innerText verwenden. DOWNLOAD_PDF eignet sich nur, wenn der Link tatsächlich den passenden POST auslöst. Der lokale PDF-Demo verwendet dafür ein Submit-Element; nicht jeder Link erfüllt diese Bedingung.

op: ASSERT — Prüfungen für Links

action	Präzise Prüfung	expected
EXISTS	Wartet, bis der ausgewählte Link im DOM vorhanden ist (attached). Sichtbarkeit ist nicht erforderlich.	Nicht erforderlich.
NOT_EXISTS	Wartet auf null Treffer.	Nicht erforderlich.

VISIBLE	Prüft, dass ein ausgewählter Link sichtbar ist.	Nicht erforderlich.
HIDDEN	Prüft Unsichtbarkeit; ein nicht vorhandenes Element erfüllt die Prüfung ebenfalls.	Nicht erforderlich.
ENABLED	Prüft den aktivierten Zustand nach Playwright-Semantik.	Nicht erforderlich.
DISABLED	Prüft den deaktivierten Zustand nach Playwright-Semantik.	Nicht erforderlich.
COUNT	Prüft die exakte Anzahl der Locator-Treffer.	Integer erforderlich, etwa 0, 1 oder 2.
TEXT	Prüft den vollständigen normalisierten Elementtext mit innerText.	String erforderlich; kein Joker.
TEXT_EQUALS	Alias von TEXT, exakt derselbe Implementierungszweig.	String erforderlich.
TEXT_CONTAINS	Prüft, ob der normalisierte Elementtext einen literalen Teilstring enthält.	String erforderlich; Sternchen ist kein Joker.
TEXT_GLOB	Prüft den vollständigen Text gegen ein Glob-Muster mit normalisierten Textleerzeichen/NBSP.	String erforderlich; * steht für beliebig viele Zeichen, auch null; ? bleibt literal.
TEXT_MATCHES	Prüft den Elementtext gegen einen regulären	Regex-String erforderlich; ^ und $

	Ausdruck mit DOTALL.	setzen vollständige Grenzen. Java-spezifische Regex-Konstrukte vermeiden, da die Browserprüfung JavaScript verwendet.

Bei normalen HTML-Links existiert kein natives disabled-Attribut wie bei Buttons. ENABLED oder DISABLED ist daher keine Garantie für fachliche Berechtigung oder erfolgreiche Navigation. ARIA-Zustände und das tatsächliche Verhalten prüfen. Eine CSS-Klasse namens disabled ist nicht automatisch ein deaktivierter Zustand.

TEXT-Prüfungen lesen den Elementtext; link.text wählt nach zugänglichem Namen. Bei aria-label können sich diese Werte unterscheiden. Beispiel: <a aria-label="Dokumentation öffnen" href="/help">Hilfe wird über den Namen Dokumentation öffnen gefunden, während TEXT Hilfe prüft.

Warum werden andere GUI-Funktionen hier nicht als Link-Aktionen erklärt?

`link` beschreibt das zu suchende Element. `action` beschreibt das Verhalten. Ein normales `<a href="...">` kann angeklickt, mit der Tastatur aktiviert oder auf Text und Sichtbarkeit geprüft werden. Es ist aber kein Eingabefeld, Dropdown oder Checkbox.

Testabsicht	Passendes Element / Kapitel	Warum nicht im Link-Kapitel?
Text eingeben: FILL	Eingabefelder, L03	Ein Link besitzt keinen editierbaren Eingabewert.
Option wählen: SELECT	Dropdowns, L04	Ein Link besitzt keine auswählbaren select-Optionen.
CHECK / UNCHECK und CHECKED / UNCHECKED	Checkboxen / Radio, L05	Ein Link besitzt keinen checked-Zustand.
VALUE / VALUE_GLOB / VALUE_MATCHES	Eingabewerte	Text eines Links ist nicht inputValue. Für Links TEXT-Prüfungen verwenden.

UPLOAD	Datei-Eingabefeld, L11	Der Dateidialog gehört zum input type=file.
OPEN / BACK / SCREENSHOT	Seite und Navigation	Diese Aktionen betreffen die Seite, nicht den ausgewählten Link.
OPEN_TAB / FIND_TAB / CLOSE_TAB / BUFFER_TAB_EXISTS	Browser-Tabs, L10	Sie verwalten Tabs. `page` ist nur ein Logtext.
CLOSE / CLOSE_BROWSER	Browser-Session	Beendet die Session, nicht einen Link.
ASSERT_PAGE_STRUCTURE	Separates Thema: ARIA-Seitenstruktur	Vergleicht eine Seitenbaseline; keine einzelne Linkprüfung.

Diese Tabelle ist eine Orientierung, keine Behauptung, dass jede allgemeine GUI-Funktion bereits in der Link-Lektion demonstriert wird. INPUT ist ein vorhandener Alias von FILL. CLOSE_BROWSER ist ein vorhandener Alias von CLOSE. Die getrennten Kapitel müssen ihre Funktionen genauso vollständig beschreiben und mit eigenen Fixtures nachweisen.

Timeout: Woher kommen 10000 und 30000 ms?

Der wirksame GUI-Timeout wird in `GuiStepConfig.resolve` festgelegt:

```
int timeout = result.get("timeout") != null ?
integer(result.get("timeout"))
        : tc.getTimeout() != null ? tc.getTimeout() : 30000;
```

Priorität	Definition	Beispiel	Log
1	timeout im einzelnen Schritt	timeout: 2500	2500 ms
2	timeout im TestCase	timeout: 10000 auf Ebene von name/steps	10000 ms, sofern der Schritt nichts überschreibt
3	Fester Standard im Java-Code	Weder TestCase noch Schritt definiert timeout	30000 ms

Der Wert 30000 ist der letzte Fallback, nicht ein zusätzlicher Wartewert. Der Wert 10000 in deinem Log ist daher korrekt: `01-links.yaml` definiert `timeout: 10000`. Die Reusable-Schritte verwenden diesen TestCase-Kontext. Ein schneller Erfolg endet sofort; timeout ist eine Obergrenze und keine Lernpause. Ein Timeoutwert muss positiv sein.

In `01-links.yaml` wird ein VISIBLE-Schritt ausdrücklich mit timeout 2500 ausgeführt. `Timeout_Default_30000.yaml` enthält dagegen keinen timeout und zeigt den Java-Fallback. So sind beide Regeln direkt im Log nachvollziehbar.

Pflichtparameter und Standardwerte

Für alle locatorbezogenen Aktionen ist genau eine link- oder target-Auswahl erforderlich. Ein Link-Locator wird nicht gleichzeitig mit target angegeben. Die oben genannten Prüfungen werden bis zum konfigurierten Timeout wiederholt bzw. abgewartet. timeout ist optional: expliziter Schrittwert, sonst TestCase-Timeout, sonst 30000 ms im geprüften Code. Unbekannte Aktionen, fehlende Pflichtwerte und nicht passende Elementtypen sind Fehler; sie werden nicht stillschweigend ignoriert.

Ein zeitbasierter Lern-WAIT ist ein eigenständiger Schritt vom Typ WAIT:

```yaml
# Fünf Sekunden Zeit zum Beobachten der Seite.
- type: WAIT
  value: '5000'
```

Ein GUI-WAIT mit Link wartet dagegen auf den gewünschten Elementzustand. Diese zwei Anwendungen nicht verwechseln.

Beispiele für jede Link-Aktion

Die folgenden Fragmente zeigen die implementierten Parameter. Sie bilden keine gemeinsame ausführbare Suite: negative Zustände und Download-/POST-Ziele benötigen passende HTML-Fixtures. Der Link L01-help und die normalen Textprüfungen beziehen sich auf die vorhandene lokale Lernseite.

EXEC / CLICK

```yaml
# Den eindeutig identifizierten Link anklicken.
- type: GUI
  op: EXEC
  action: CLICK
  link:
    id: L01-help
```

EXEC / HOVER

```yaml
# Den Mauszeiger auf den Link bewegen.
- type: GUI
  op: EXEC
  action: HOVER
  link:
    id: L01-help
```

EXEC / PRESS

```yaml
# Den Link über die Tastatur aktivieren.
```

```
- type: GUI
  op: EXEC
  action: PRESS
  link:
    id: L01-help
  value: Enter
```

EXEC / WAIT

```
# Bis zur Sichtbarkeit warten; dies ist keine feste Pause.
- type: GUI
  op: EXEC
  action: WAIT
  link:
    id: L01-help
  state: visible
  timeout: 5000
```

EXEC / EXTRACT

```
# Das href lesen und in einem Buffer speichern.
- type: GUI
  op: EXEC
  action: EXTRACT
  link:
    id: L01-menu
  attribute: href
  buffer: DemoLinkHref
```

EXEC / DOWNLOAD

```
# Benötigt die Seite 11-files.html und einen echten Downloadlink.
- type: GUI
  op: EXEC
  action: DOWNLOAD
  target:
    css: '#L11-download'
  value: learning-download.txt
  buffer: DemoDownload
  timeout: 10000
```

EXEC / DOWNLOAD_PDF

```
# POST-PDF-Beispiel: Die Seite 11-files.html verwendet ein Submit-
Element, keinen normalen GET-Link.
- type: GUI
  op: EXEC
  action: DOWNLOAD_PDF
  target:
    css: '#L11-pdf'
  outputFile: output/learningdemo/original.pdf
  responseUrlGlob: '**/dbasQuittung.xhtml*'
  buffer: DemoPDF
  timeout: 10000
```

ASSERT / EXISTS

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: EXISTS
  link:
    id: L01-help
```

ASSERT / NOT_EXISTS

```
# Ein bewusst nicht vorhandenes Element prüfen.
- type: GUI
  op: ASSERT
```

```
  action: NOT_EXISTS
  link:
    id: nicht-vorhandener-link
```

ASSERT / VISIBLE

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: VISIBLE
  link:
    id: L01-help
```

ASSERT / HIDDEN

```
# Ein bewusst nicht vorhandenes Element prüfen.
- type: GUI
  op: ASSERT
  action: HIDDEN
  link:
    id: nicht-vorhandener-link
```

ASSERT / ENABLED

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: ENABLED
  link:
    id: L01-help
```

ASSERT / DISABLED

```
# Zusätzliche Fixture mit wirksamer Deaktivierung erforderlich; kein positiver L01-Test.
- type: GUI
  op: ASSERT
  action: DISABLED
  link:
    id: deaktivierter-link
```

ASSERT / COUNT

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: COUNT
  link:
    text: (H)elp
    exact: false
  expected: 2
```

ASSERT / TEXT

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: TEXT
  link:
    id: L01-help
  expected: (H)elp
```

ASSERT / TEXT_EQUALS

```
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_EQUALS
```

```
  link:
    id: L01-help
  expected: (H)elp
```

ASSERT / TEXT_CONTAINS

```yaml
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_CONTAINS
  link:
    id: L01-help
  expected: elp
```

ASSERT / TEXT_GLOB

```yaml
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_GLOB
  link:
    id: L01-help
  expected: '*elp'
```

ASSERT / TEXT_MATCHES

```yaml
# Den Link prüfen.
- type: GUI
  op: ASSERT
  action: TEXT_MATCHES
  link:
    id: L01-help
  expected: ^\(H\)elp$
```

AGATE Best Practice

Zuerst die fachliche Absicht bestimmen: klicken, lesen, warten oder prüfen. Danach action und die notwendigen Parameter wählen. Nach CLICK oder PRESS eine passende Zielseiten-/Zustandsprüfung ergänzen. COUNT ist ausdrücklich für Mehrfachtreffer geeignet; ein mehrdeutiger CLICK wird nicht durch exact false repariert. Für DOWNLOAD nur tatsächliche Downloads verwenden, für DOWNLOAD_PDF den vorhandenen POST-Vertrag prüfen. Prüfungen von Linknamen und Prüfungen von sichtbarem Text getrennt beschreiben.

Parameter von link

Parameter	Typ	Pflicht / Standard	Bedeutung
text	Nicht leerer String	Genau eines von text oder id	Zugänglicher Name eines Elements mit Link-Rolle.
id	Nicht leerer String	Genau eines von text oder id	HTML-ID; wird zu a[id="..."]. Nicht mit der

			Schritt-ID verwechseln.
exact	Boolean	Optional bei text; Standard true	true: vollständiger normalisierter Name, Groß-/Kleinschreibung relevant. false: Teilstring, Groß-/Kleinschreibung nicht relevant.
index	Nicht negativer Integer	Optional	Position innerhalb der Treffer; beginnt bei 0.

Bei Linkid — Find the Best Internships & Jobs darf exact nicht angegeben werden. Die ID-Auswahl ist immer exakt. text und id dürfen nicht gleichzeitig verwendet werden. exact: 'true' ist ein String; korrekt ist exact: true. Ebenso ist index: 0 korrekt und index: '0' nicht.

Was bedeutet exact?

Die Seite enthält (H)elp und (H)elp overview:

Auswahl	Ergebnis
text (H)elp, exact true	Nur (H)elp.
text (H)elp, exact weggelassen	Dasselbe: AGATE verwendet true.
text (H)elp, exact false	Beide Links; CLICK ist mehrdeutig. COUNT kann 2 prüfen.
text (h)elp, exact true	Andere Groß-/Kleinschreibung: kein Treffer.
text (h)elp, exact false	Teilstringvergleich ohne Beachtung der Groß-/Kleinschreibung.
text (H)elp*, exact true	Sucht eine tatsächliche Sternchen-Beschriftung; kein Joker.

Auch exact true kann mehrere Treffer liefern, wenn mehrere Links denselben Namen haben. exact false bedeutet weder „ähnlichster Treffer“ noch „erster Treffer“.

Technischer target-Locator

Feld	Verwendung	Beispiel
css	CSS-Auswahl	a[id="L01-help"]
role, name, exact	Rolle und zugänglicher Name	role link, name (H)elp, exact true
testId	data-testid	help-link
text, exact	Textauswahl ohne Beschränkung auf Links	(H)elp overview
linkText, exact	Technische Form der Link-Rollenauswahl	(H)elp
scope	Bereich als weiteres target-Objekt	css #navigation
frames	Iframe-CSS-Selektoren von außen nach innen	['#help-frame']
nth	Nultbasierter Trefferindex	0

Genau eine Strategie aus css, role, testId, text, linkText, label oder placeholder wählen. name ergänzt role; scope, frames und nth ergänzen die Strategie. Diese drei Felder sind innerhalb der vereinfachten link-Form nicht erlaubt. Dafür target verwenden. exact verändert die CSS-Syntax nicht.

link.href, link.regex und link.textGlob sind keine unterstützten Parameter. Nicht alle Optionen des direkten Playwright-API sind als AGATE-YAML verfügbar. Insbesondere ist der AGATE-Standard exact true nicht mit jedem Playwright-Standard identisch.

Vollständiges Beispiel für die lokale Lernseite

Die HTML-Beschriftungen bleiben unverändert, damit die Beispiele mit dem vorhandenen Demo funktionieren. Eine Übersetzung des Locator-Textes ohne Änderung der HTML-Seite wäre ein Fehler.

```
name: Links – Exakte Auswahl und alternative Locators
steps:
```

```yaml
  - id: open-links
    type: GUI
    op: EXEC
    action: OPEN
    url: 'http://127.0.0.1:8767/01-links.html'

  # Der verschachtelte span benötigt keinen eigenen Locator.
  - id: help-by-name
    type: GUI
    op: ASSERT
    action: VISIBLE
    page: L01 — Links
    link:
      text: '(H)elp'
      exact: true

  # Der Teilstring findet beide Links. Deshalb die Anzahl prüfen.
  - id: help-substring-count
    type: GUI
    op: ASSERT
    action: COUNT
    link:
      text: '(H)elp'
      exact: false
    expected: 2

  # HTML-ID: exact muss weggelassen werden.
  - id: help-by-id
    type: GUI
    op: ASSERT
    action: VISIBLE
    link:
      id: L01-help

  # Derselbe Link mit CSS.
  - id: help-by-css
    type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      css: 'a[id="L01-help"]'

  # Derselbe Link über data-testid.
  - id: help-by-testid
    type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      testId: help-link

  # Derselbe Link mit expliziter Rolle und zugänglichem Namen.
  - id: help-by-role
    type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      role: link
      name: '(H)elp'
      exact: true

  # Nach der Navigation auch die Zielseite prüfen.
  - id: next-lesson
    type: GUI
    op: EXEC
    action: CLICK
    link:
      text: 'Next lesson: buttons'
      exact: true
  # Lernpause: keine Synchronisationsstrategie für Regressionstests.
  - type: WAIT
    value: '5000'
  - id: destination-visible
```

```
    type: GUI
    op: ASSERT
    action: VISIBLE
    target:
      css: '#L02-title'
```

Gleicher Name in zwei Bereichen

Dieses zusätzliche HTML ist nicht Bestandteil der L01-Lernseite:

```
<nav id="navigation"><a href="/help">Hilfe</a></nav>
<footer><a href="/help">Hilfe</a></footer>
```

```
- type: GUI
  op: EXEC
  action: CLICK
  target:
    role: link
    name: Hilfe
    exact: true
    scope:
      css: '#navigation'
```

Link in einem Iframe

Die getestete Seite muss das genannte Iframe tatsächlich enthalten:

```
- type: GUI
  op: ASSERT
  action: VISIBLE
  target:
    role: link
    name: Hilfe
    exact: true
    frames:
      - '#help-frame'
```

Auswahl über href

```
- type: GUI
  op: EXEC
  action: CLICK
  target:
    css: 'a[href="/02-buttons.html"]'
```

Der DOM-Attributwert muss exakt passen. Absolute URLs, Session-IDs und zusätzliche Parameter können diese Auswahl verändern.

Attribut lesen und Text prüfen

```
# EXTRACT liest das Attribut; es prüft dessen Inhalt nicht automatisch.
- type: GUI
  op: EXEC
  action: EXTRACT
  link:
    id: L01-menu
  attribute: href
  buffer: DemoLinkHref
```

VISIBLE prüft die Sichtbarkeit, EXISTS das Vorhandensein und COUNT die Anzahl. TEXT, TEXT_CONTAINS und TEXT_GLOB prüfen den Text des gewählten Elements. Dieser muss bei ARIA-Attributen nicht dem zugänglichen Namen entsprechen. In link.text ist * kein Joker;

im expected einer TEXT_GLOB-Prüfung ist * ein Joker. ? ist in der geprüften AGATE-Glob-Implementierung ein normales Zeichen.

Fehler systematisch untersuchen

Symptom	Prüfen	Maßnahme
Kein Treffer	URL, Name, Groß-/Kleinschreibung, ARIA, Iframe	Locator im genannten YAML-Schritt korrigieren.
Mehrere Treffer	Doppelte Namen und COUNT	Scope oder stabile ID verwenden.
CLICK-Timeout	Vorhandensein, Sichtbarkeit, Overlay, Zustand	Screenshot und DOM prüfen. Timeout allein beweist keinen falschen Text.
exact mit id abgelehnt	Parameterkombination	exact entfernen.
Nicht unterstütztes Feld	href/regex/textGlob innerhalb link	Passende target-Strategie verwenden.
Falsche aktive Seite	Navigation und neue Tabs	Tabverwaltung prüfen; page ist nur eine Beschreibung.

Vollständige ausführbare Link-Lektion im Demo

Die bisherige kurze Locator-Demonstration wurde erweitert. `01-links.yaml` und `Links_Learning.yaml` rufen dieselbe vollständige Datei `reusable/01-links.yaml` auf. Die L01-Seite enthält nun auch die benötigten Zustands- und Download-Fixtures.

EXEC	Konkretes Verhalten in L01
CLICK	Help startet verzögerte Anzeige; ein anderer Link entfernt sich selbst.
HOVER	Ein Hinweis wird eingeblendet und anschließend geprüft.
PRESS	Enter aktiviert den Link und setzt einen überprüfbaren Ergebnistext.

WAIT	Wartet unmittelbar nach dem Trigger auf den verzögerten Link.
EXTRACT	Liest href, speichert L01Href und verwendet den Buffer in einer DOM-Prüfung.
DOWNLOAD	Lädt eine reale Textdatei herunter.
DOWNLOAD_PDF	Der Link sendet einen Formular-POST; die originale PDF-Antwort wird gespeichert.

Alle zwölf linkgeeigneten ASSERT-Werte besitzen positive Beispiele: EXISTS, NOT_EXISTS, VISIBLE, HIDDEN, ENABLED, DISABLED, COUNT, TEXT, TEXT_EQUALS, TEXT_CONTAINS, TEXT_GLOB und TEXT_MATCHES. DISABLED verwendet ausdrücklich aria-disabled=true. Die absichtlich fehlschlagenden Tests bleiben im Verzeichnis negative.

DOWNLOAD_PDF ist ein Spezialfall: Der Link verhindert seine normale GET-Navigation und löst requestSubmit an einem POST-Formular aus. Die Fixture erklärt damit den vorhandenen Serververtrag; sie suggeriert keine allgemeine PDF-GET-Unterstützung.

Lernpausen sind fünf Sekunden lang. Zwischen dem Trigger für die verzögerte Anzeige und GUI WAIT steht bewusst keine feste Pause: Sonst wäre das zustandsabhängige Warten nicht demonstriert. Abschnittstitel und Beschreibungen sind deutsch; HTML-Namen, die vorhandene Locators benötigen, bleiben unverändert.

Vollständiger Reusable-Inhalt

```
# Vollständige Link-Lektion: EXEC-Aktionen und alle linkgeeigneten ASSERT-Werte.
steps:
- section: Vorbereitung
  description: Die Link-Lernseite mit definiertem Ausgangszustand öffnen.
- id: l01-1
  type: GUI
  op: EXEC
  action: OPEN
  page: L01 - Links
  url: http://127.0.0.1:8767/01-links.html
- type: WAIT
  value: '5000'
- section: Locator - Exakter Name
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-3
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    text: (H)elp
    exact: true
```

```
- section: Locator - Standard exact=true
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-4
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    text: (H)elp
- section: Locator - HTML-ID
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-5
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    id: L01-help
- section: Locator - CSS
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-6
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  target:
    css: '#L01-help'
- section: Locator - testId
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-7
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  target:
    testId: help-link
- section: Locator - Rolle und Name
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-8
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  target:
    role: link
    name: (H)elp
    exact: true
- section: Locator - Text-Locator
  description: Denselben Link über die erklärte Strategie auswählen; VISIBLE prüft
    den Zustand.
- id: l01-9
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  target:
    text: (H)elp overview
    exact: true
- section: Teilstring und COUNT
  description: exact=false findet beide Help-Links. COUNT prüft die Anzahl ohne mehrdeutigen
```

```
    Klick.
- id: l01-10
  type: GUI
  op: ASSERT
  action: COUNT
  page: L01 - Links
  link:
    text: (H)elp
    exact: false
  expected: 2
- section: Timeout auf Schrittebene
  description: Dieser Schritt überschreibt den TestCase-Wert 10000
mit 2500 ms.
- id: l01-11
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    id: L01-help
  timeout: 2500
- section: ASSERT - TEXT
  description: Den Elementtext prüfen. TEXT_EQUALS ist ein Alias von
TEXT; nur TEXT_GLOB
    verwendet den Joker *.
- id: l01-12
  type: GUI
  op: ASSERT
  action: TEXT
  page: L01 - Links
  link:
    id: L01-help
  expected: (H)elp
- section: ASSERT - TEXT_EQUALS
  description: Den Elementtext prüfen. TEXT_EQUALS ist ein Alias von
TEXT; nur TEXT_GLOB
    verwendet den Joker *.
- id: l01-13
  type: GUI
  op: ASSERT
  action: TEXT_EQUALS
  page: L01 - Links
  link:
    id: L01-help
  expected: (H)elp
- section: ASSERT - TEXT_CONTAINS
  description: Den Elementtext prüfen. TEXT_EQUALS ist ein Alias von
TEXT; nur TEXT_GLOB
    verwendet den Joker *.
- id: l01-14
  type: GUI
  op: ASSERT
  action: TEXT_CONTAINS
  page: L01 - Links
  link:
    id: L01-help
  expected: elp
- section: ASSERT - TEXT_GLOB
  description: Den Elementtext prüfen. TEXT_EQUALS ist ein Alias von
TEXT; nur TEXT_GLOB
    verwendet den Joker *.
- id: l01-15
  type: GUI
  op: ASSERT
  action: TEXT_GLOB
  page: L01 - Links
  link:
    id: L01-help
  expected: '*elp'
- section: ASSERT - TEXT_MATCHES
  description: Den Elementtext prüfen. TEXT_EQUALS ist ein Alias von
TEXT; nur TEXT_GLOB
```

```
    verwendet den Joker *.
- id: l01-16
  type: GUI
  op: ASSERT
  action: TEXT_MATCHES
  page: L01 - Links
  link:
    id: L01-help
  expected: ^\(H\)elp$
- section: Leerzeichen und NBSP
  description: Die Namensauswahl normalisiert Leerzeichen; exact
verlangt keine Rohtext-Gleichheit.
- id: l01-17
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    text: Leerzeichen normalisiert
    exact: true
- section: ASSERT - EXISTS
  description: Ein unsichtbarer Link kann trotzdem im DOM vorhanden
sein.
- id: l01-18
  type: GUI
  op: ASSERT
  action: EXISTS
  page: L01 - Links
  link:
    id: L01-hidden
- section: ASSERT - HIDDEN
  description: Den vorhandenen, aber unsichtbaren Link prüfen.
- id: l01-19
  type: GUI
  op: ASSERT
  action: HIDDEN
  page: L01 - Links
  link:
    id: L01-hidden
- section: ASSERT - ENABLED
  description: Den normalen Linkzustand prüfen; dies garantiert noch
keine fachliche
    Berechtigung.
- id: l01-20
  type: GUI
  op: ASSERT
  action: ENABLED
  page: L01 - Links
  link:
    id: L01-help
- section: ASSERT - DISABLED
  description: Diese Fixture besitzt aria-disabled=true; ein bloßes
CSS-disabled wäre
    nicht gleichwertig.
- id: l01-21
  type: GUI
  op: ASSERT
  action: DISABLED
  page: L01 - Links
  link:
    id: L01-disabled
- section: EXEC - HOVER
  description: HOVER blendet den Hinweis ein; danach wird sein Text
geprüft.
- id: l01-22
  type: GUI
  op: EXEC
  action: HOVER
  page: L01 - Links
  link:
    id: L01-hover
- type: WAIT
```

```yaml
  value: '5000'
- section: Hover-Ergebnis
  description: Den durch HOVER eingeblendeten Hinweis prüfen.
- id: l01-24
  type: GUI
  op: ASSERT
  action: TEXT
  page: L01 - Links
  target:
    css: '#L01-tooltip'
  expected: Hinweis nach HOVER
- section: EXEC - PRESS
  description: Enter aktiviert einen fokussierten Link. Die Fixture
protokolliert
    die Aktivierung.
- id: l01-25
  type: GUI
  op: EXEC
  action: PRESS
  page: L01 - Links
  link:
    id: L01-key
  value: Enter
- type: WAIT
  value: '5000'
- section: Tastatur-Ergebnis
  description: Das Ergebnis der Enter-Aktivierung prüfen.
- id: l01-27
  type: GUI
  op: ASSERT
  action: TEXT
  page: L01 - Links
  target:
    css: '#L01-key-result'
  expected: Enter hat den Link aktiviert
- section: EXEC - CLICK
  description: Der Help-Klick startet die verzögerte Anzeige.
- id: l01-28
  type: GUI
  op: EXEC
  action: CLICK
  page: L01 - Links
  link:
    id: L01-help
- section: EXEC - WAIT
  description: Auf visible warten, höchstens 2000 ms; keine feste
Zwei-Sekunden-Pause.
- id: l01-29
  type: GUI
  op: EXEC
  action: WAIT
  page: L01 - Links
  link:
    id: L01-delayed
  state: visible
  timeout: 2000
- section: WAIT-Ergebnis
  description: Der verzögerte Link muss nun sichtbar sein.
- id: l01-30
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    id: L01-delayed
- section: EXEC - EXTRACT
  description: href des Menülinks als L01Href lesen. EXTRACT allein
ist keine Prüfung.
- id: l01-31
  type: GUI
  op: EXEC
  action: EXTRACT
```

```
  page: L01 - Links
  link:
    id: L01-menu
  attribute: href
  buffer: L01Href
- type: WAIT
  value: '5000'
- section: EXTRACT-Ergebnis
  description: Den gelesenen href für eine konkrete DOM-Prüfung
verwenden.
- id: l01-33
  type: GUI
  op: ASSERT
  action: COUNT
  page: L01 - Links
  target:
    css: a[id="L01-menu"][href="{B[L01Href]}"]
  expected: 1
- section: Entfernen per CLICK
  description: Dieser Link entfernt sich beim Klick aus dem DOM.
- id: l01-34
  type: GUI
  op: EXEC
  action: CLICK
  page: L01 - Links
  link:
    id: L01-remove
- type: WAIT
  value: '5000'
- section: ASSERT - NOT_EXISTS
  description: Nach der Entfernung darf kein Treffer mehr vorhanden
sein.
- id: l01-36
  type: GUI
  op: ASSERT
  action: NOT_EXISTS
  page: L01 - Links
  link:
    id: L01-remove
- section: EXEC - DOWNLOAD
  description: Ein echtes Browser-Downloadereignis erzeugen und die
Textdatei speichern.
- id: l01-37
  type: GUI
  op: EXEC
  action: DOWNLOAD
  page: L01 - Links
  link:
    id: L01-download
  value: learning-link.txt
  buffer: L01Download
  timeout: 10000
- type: WAIT
  value: '5000'
- section: EXEC - DOWNLOAD_PDF
  description: Dieser Link startet einen POST. Originalantwort
speichern; kein HTML-Druck
    und kein allgemeiner GET-Downloader.
- id: l01-39
  type: GUI
  op: EXEC
  action: DOWNLOAD_PDF
  page: L01 - Links
  link:
    id: L01-pdf
  outputFile: output/learningdemo/link-original.pdf
  responseUrlGlob: '**/dbasQuittung.xhtml*'
  buffer: L01PDF
  timeout: 10000
- type: WAIT
  value: '5000'
- section: Nach PDF zurück auf der Lernseite
```

```
  description: DOWNLOAD_PDF schließt neue PDF-Popups; der
ursprüngliche Link bleibt
    erreichbar.
- id: l01-41
  type: GUI
  op: ASSERT
  action: VISIBLE
  page: L01 - Links
  link:
    id: L01-pdf
```